

Hier ist die Analyse des PDF-Auszugs mit den Aufgaben und Lösungen:

Aufgabe 1: Fehlersuche

** (a) Fehler: `true` statt `True` **

- **Fehler:** `bestanden = true` (Python ist case-sensitive, `True` muss großgeschrieben sein).
- **Korrektur:** `bestanden = True`

** (b) Fehler: Längeninkonsistenz zwischen Listen **

- **Fehler:** `stadt` hat 4 Elemente, `einwohner` nur 3 → `IndexError` bei `einwohner[3]`.
- **Korrektur:** Listen müssen gleich lang sein oder `einwohner` muss um ein Element ergänzt werden

** (c) Fehler: Kein Basisfall in rekursiver Fakultätsfunktion **

- **Fehler:** Keine Abbruchbedingung für `n = 0` oder `n = 1` → unendliche Rekursion.
- **Korrektur:** `if n <= 1: return 1` hinzufügen.

** (d) Fehler: Uninitialisierte Variable `i` **

- **Fehler:** `i` wird nicht definiert, bevor es in der `while`-Schleife verwendet wird.
- **Korrektur:** `i = 0` vor der Schleife initialisieren.

** (e) Fehler: Division durch Null **

- **Fehler:** `b = 0.6` wird zu `int(0.6) = 0`, was zu einer Division durch Null führt.
- **Korrektur:** `b` auf einen Wert $\neq 0$ setzen (z. B. `b = 1`).

Aufgabe 2: Ausgabeanalyse

** (a) Ausgabe: `True` **

- `vier \* vier == zwei` → `2 \* 2 == 4` → `4 == 4` → `True`.

** (b) Ausgabe: `3` **

- `range(3, 4)` → nur `i = 3` wird ausgegeben.

** (c) Ausgabe: `i`, `j`, `k` **

- Die `while`-Schleife durchläuft die Liste `l` und gibt jedes Element aus.

** (d) Ausgabe: `allesklar` **

- Die Indizes entsprechen den Buchstaben: `Rekursionmacht allen Spaß` → `a`, `l`, `l`, `e`, `m`, `a`, `c`.

** (e) Ausgabe: `1 2 3 4 5` **

- Die Funktion `k(i)` druckt `i` und ruft sich rekursiv mit `i+1` auf, bis `i > 5`.

Aufgabe 3: Run Length Encoding (RLE)

** (a) `encode(text)` **

- **Lösung:**

```
```python
```

```
def encode(text):
```

```
 result = []
```

```
 count = 1
```

```

for i in range(1, len(text)):
 if text[i] == text[i-1] and count < 9:
 count += 1
 else:
 result.append(f"{count}{text[i-1]}")
 count = 1
result.append(f"{count}{text[-1]}")
return "".join(result)
...

```

##### **\*\*(b) `decode(text)`**

- **\*\*Lösung:\*\***

```

```python
def decode(text):
    result = []
    i = 0
    while i < len(text):
        if text[i].isdigit():
            count = int(text[i])
            result.append(text[i+1] * count)
            i += 2
        else:
            result.append(text[i])
            i += 1
    return "".join(result)
...

```

****(c) RLE ineffizient bei:****

- ****Falls:**** Der Text bereits sehr komprimiert ist (z. B. `ABCABCABC` → `1A1B1C1A1B1C`).
- ****Oder:**** Die Wiederholungen kurz sind (z. B. `ABABAB` → `1A1B1A1B1A1B`).

****(d) `checkRLE(text)`**

- ****Lösung:****

```

```python
def checkRLE(text):
 original_length = len(text)
 encoded = encode(text)
 return len(encoded) < original_length
...

```

---

### **\*\*Aufgabe 4: Schimpfwortfilter\*\***

##### **\*\*(a) `readSwearwordData(filename)`**

- **\*\*Lösung:\*\***

```

```python
def readSwearwordData(filename):
    try:
        with open(filename, 'r') as file:
            return [line.strip().split() for line in file]
    except:
        pass

```

```

except:
    return []
...

```

```
##### **(b) `checkText(message, swearwords)`**
```

```
- **Lösung:**
```

```
```python
```

```
def checkText(message, swearwords):
 total_severity = 0
 words = message.split()
 for i, word in enumerate(words):
 for entry in swearwords:
 if word.lower() == entry[0].lower():
 total_severity += int(entry[1])
 words[i] = "****"
 if total_severity > 7:
 return "Nachricht wurde gelöscht"
 return " ".join(words)
...

```

```
(c) Hauptprogramm
```

```
- **Lösung:**
```

```
```python
```

```
print(checkText("Du bist natürlich kein Idiot", readSwearwordData("swearwords.txt")))
...

```

```
---
```

```
#### **Aufgabe 5: Java-Klasse `Moebel`**
```

```
- **Lösung:**
```

```
```java
```

```
public class Moebel {
 private String name;
 private double preis;

 public Moebel(String name, double preis) {
 this.name = name;
 this.preis = preis;
 }

 @Override
 public String toString() {
 return name + " (" + preis + " €)";
 }
}
...

```

```

```

```
Zusammenfassung:
```

- **Fehleranalyse:** Typische Python-Fehler (Groß-/Kleinschreibung, uninitialisierte Variablen, Division
- **RLE:** Komprimierung mit Begrenzung auf 9 Wiederholungen.
- **Schimpfwortfilter:** Einlesen von Wörtern, Zensierung und Schweregradprüfung.
- **Java:** Klasse mit Konstruktor und `toString()`-Methode.

Falls weitere Details benötigt werden, gerne nachfragen!